Implementation and Evaluation of the 1990 Simplified Jordan Sorting Algorithm

Master's Thesis

Review draft

16 October 2026

Freie Universität Berlin
Department of Mathematics and Computer Science
Institute of Computer Science
Theoretical Computer Science

For supervisor review

Current implementation and
matched experiments

Abstract

Jordan sorting recovers the coordinate order of intersections along a line from their traversal order on a simple curve. This thesis reconstructs the 1990 Simplified Jordan Sorting procedure as one executable Python core with ordinary-list and heterogeneous red–black finger-tree backends. The core maintains its own partial sorted order, upper/lower pair families and sibling ownership; external input certification supplies the validity precondition, not the sorted output. Update errors propagate without rollback.

The tree implements stable pair handles, endpoint fingers, local insertion, split and header-based ownership transfer. A source-level manual argument checks its physical representation, locality and mixed-operation potentials. Under unit-cost comparison and machine-word assumptions, with expected lookup and amortized dictionary growth, it gives linear total work for the valid-input minimal core. This is not machine verification or an unconditional Python wall-clock guarantee. Independent model checks, bounded validation and same-backend replay provide separate execution evidence.

The current matched study qualifies 140 controlled inputs at lengths 32–513 and a separate twelve-input set with two-sided splits. Frozen per-input batch sizes and twenty paired batches per backend yield 6,080 formal sample batches. All 66,560 sample calls return correct outputs. Tree/list ratios of input medians range from 1.114 to 3.535 in the common set (median 2.898), and from 3.147 to 3.213 in the targeted set (median 3.180). The tree is slower on every measured input; no future crossover is inferred. Related constructions and mirrors are not independent random samples.

The contribution combines an independently recovering implementation, source-linked representation and cost analysis, and reproducible matched measurements with explicit timing and evidence boundaries. The earlier rollback-enabled 60-input study remains separately identified historical evidence. Recognition and the causal effect of rollback removal are not measured. The results distinguish theoretical operation costs from the practical constants of the implemented Python backends.

Contents

Abstract . . . . . **i**

List of Figures . . . . . **vii**

List of Tables . . . . . **ix**

1. Introduction **1**
1.1. Motivation 1
1.2. Problem and Scope 1
1.3. Research Questions 2
1.4. Contributions 2
1.5. Headline Findings 3
1.6. Thesis Structure 3

2. Background **5**
2.1. Jordan Sequences and the Sorting Problem 5
2.2. The Structural Restriction Behind Jordan Sorting 6
2.3. The 1986 Linear-Time Algorithm 7
2.4. The 1990 Simplification 8
2.5. Related Work and Positioning 8
2.5.1. Ordered Lists and Finger Search 8
2.5.2. Jordan Sorting and Geometric Applications 9
2.6. Position and Scope of This Thesis 9

3. Algorithm Reconstruction **11**
3.1. Purpose and Reconstruction Boundary 11
3.2. Pairs, Intervals, and the Two Families 12
3.2.1. Families and Sibling Lists 12
3.3. Maintained State and Prefix Invariant 13
3.4. Initialization 13

3.5. Boundary Selection . . . 14
3.5.1. Step 1: Predecessor-Side Boundary . . . 14
3.5.2. Step 2: Successor-Side Boundary . . . 14
3.6. Step 3: Increasing Orientation . . . 14
3.6.1. Step 3(a): Insert the New Pair . . . 15
3.6.2. Step 3(b): Split Off Newly Enclosed Children . . . 15
3.6.3. Step 3(c): Insert the New Point . . . 15
3.7. Step 3: Decreasing Orientation . . . 15
3.8. Executable Anchor Clarifications . . . 16
3.8.1. Curve Endpoint Versus Geometric Endpoint . . . 16
3.8.2. Odd-Index z_1 Output Anchor . . . 17
3.9. Ownership Transfer and Failure Semantics . . . 17
3.10. End-to-End Procedure . . . 19
3.11. Correctness and Complexity Boundaries . . . 21

4. Implementation . . . 23
4.1. Implementation Scope and Versions . . . 23
4.2. Data Representation . . . 23
4.2.1. Partial Sorted Order . . . 24
4.2.2. Sibling Lists and Ownership . . . 24
4.3. Core–Backend Interface and Update Semantics . . . 24
4.3.1. Initialization, Boundaries and Stage State . . . 24
4.3.2. Pair Insertion and Failure Propagation . . . 25
4.3.3. Split Plans and Ownership Transfer . . . 25
4.3.4. Point Insertion and Completion . . . 26
4.4. Certification, Diagnostics and Replay . . . 26
4.5. Execution Policies and Observation Costs . . . 27
4.6. Actual Costs and Complexity Boundary . . . 28

5. Experimental Method . . . 31
5.1. Study Scope and Version Boundary . . . 31
5.2. Current Inputs and Qualification . . . 31
5.2.1. Historical Input Protocol . . . 32
5.3. Compared Implementations and Call Scopes . . . 32
5.3.1. Python Sort . . . 32
5.3.2. Reference Pipeline . . . 32
5.3.3. Historical Ordinary-List Core . . . 33

5.4. Timing, Calibration and Scheduling . . . 33
5.4.1. Scheduling and Noise . . . 33
5.4.2. Recorded Environment . . . 34
5.5. Aggregation and Statistical Units . . . 34
5.6. Evidence Binding and Acceptance . . . 34
5.6.1. Fail-Closed Validation . . . 36
5.7. Interpretation Boundary . . . 36

6. Results . . . 37
6.1. Current Evidence and Correctness . . . 37
6.2. Common-Set Runtime and Paired Ratios . . . 37
6.2.1. Structural Strata . . . 39
6.3. Independent Targeted Split Set . . . 39
6.4. Variability and Measurement Accounting . . . 40
6.5. Historical Rollback-Enabled Evidence . . . 40
6.6. Interpretation Boundary . . . 41

7. Limitations . . . 43
7.1. Scope and Version . . . 43
7.2. Mathematical Model and Backend Fidelity . . . 43
7.3. Input Coverage and Scale . . . 44
7.4. Timing Scopes and Environment . . . 44
7.5. Statistical and Exploratory Reach . . . 45
7.6. Output Provenance and Recognition . . . 45
7.7. Consequences for the Conclusions . . . 45

8. Conclusion . . . 47
8.1. Summary . . . 47
8.2. Answers to the Research Questions . . . 47
8.2.1. RQ1: Executable Reconstruction and Correctness . . . 47
8.2.2. RQ2: Observed Runtime Behavior . . . 48
8.2.3. RQ3: Structure and Implementation Cost . . . 48
8.3. Contributions and Remaining Work . . . 48
8.4. Closing Perspective . . . 49

A. Claim Traceability . . . 51
A.1. Claim Groups and Thesis Locations . . . 51
A.2. Implementation Revision and Evidence Scope . . . 52
A.2.1. Current Formal Evidence . . . 53

A.3. Archived Evidence Bindings 53
A.4. Interpretation Boundaries 54

Bibliography **55**

List of Figures

2.1. Jordan sorting from curve order to axis order. 6

3.1. Step 3(c) output-anchor correction for z_1. 18
3.2. Running example of the ordinary-list reconstruction. 20

4.1. Pair families and sibling-list ownership. 25

5.1. Historical formal pipeline and timing scopes. 35

6.1. Current common set. Left: medians of input medians, with a logarithmic time axis. Right: all fourteen input ratios per length; horizontal marks are their medians. Horizontal offsets only separate points. The dashed line denotes equal runtime, not a confidence bound. 38
6.2. All targeted-set input ratios. M denotes the mirrored construction. Points are ratios of twenty-batch medians, not individual samples or independent random draws. . . 40

List of Tables

3.1. Increasing and decreasing Step 3 roles in the executable reconstruction. 16

6.1. Current formal execution. Calls within a batch are not independent samples. 37
6.2. Common set: descriptive runtime and matched ratios. Times are milliseconds per call; each input first contributes a median of twenty batch-derived observations. . . 38
6.3. Common-set ratios grouped by the frozen nesting classification. These aggregates mix lengths and related constructions. 39
6.4. Targeted set, separately analyzed. Each k group has both lengths and both directions. 39
6.5. Historical rollback-enabled study only: median within-input paper/reference ratios. The two pipelines have different timed scopes. 41

A.1. Controlled claim groups and their principal locations in the thesis. 51
A.2. Archived run manifests used by the thesis analysis. 54

1. Introduction

1.1. Motivation

Jordan sorting starts with more information than arbitrary sorting: the input keys are line intersections encountered while traversing a simple plane curve. The absence of self-intersections restricts how consecutive segments can nest, permitting specialized linear-time algorithms (Hoffmann et al. 1986; Fung et al. 1990). It is a computational geometry example of turning geometric restrictions into algorithmic structure (Berg et al. 2008).

The published bound combines an incremental sorting procedure with specialized dynamic lists. The 1986 algorithm uses level-linked search trees; the 1990 simplification uses heterogeneous finger trees. Ordinary Python lists can preserve the high-level operations while imposing different costs. Moreover, the concise paper leaves operational choices about orientation, ownership, geometric endpoints and the exceptional first point to be made explicit.

This thesis reconstructs the 1990 Step 1/2/3 procedure in a shared Python core, then implements both ordinary-list and real heterogeneous red–black finger-tree storage. It connects physical representation and mixed-operation analysis to measured behavior. This follows the algorithm-engineering view that implementation, analysis and experimentation support different claims (Sanders 2009). Measurements characterize concrete programs and cases rather than replacing proof (McGeoch 2012).

1.2. Problem and Scope

For a valid Jordan sequence $z_1, \ldots, z_n$, the task is to recover increasing coordinate order from curve-traversal order. The model is introduced in chapter 2, and the executable reconstruction in chapter 3. Both backends maintain their own partial sorted order, upper/lower pair families and sibling ownership. The paper core accepts a pre-certified valid input and never consumes the oracle's sorted answer. The separate reference pipeline deliberately returns oracle-derived output; its role in the historical study has a different provenance and timed scope.

The current backends propagate update errors without rollback and require failed state to be discarded. The tree uses stable handles, endpoint fingers, local split/join and header ownership transfer. Its source-level manual argument covers representation preservation, locality and amortized cost under an explicit computational model. That argument is not machine formalization or an independent external proof certification.

The current empirical scope is valid-input sorting: 140 controlled common inputs and an independent twelve-input targeted split set, with matched timing of both minimal calls. The old 60-input rollback-enabled study is retained as historical evidence. Recognition, invalid inputs, error correction, polygon clipping and a causal experiment on rollback removal are outside scope.

1.3. Research Questions

RQ1: **Executable reconstruction and correctness.** Can one shared incremental procedure recover its own sorted output with both representations, while maintaining physical, ownership and family invariants?

RQ2: **Observed runtime behavior.** How do the two complete minimal sorting calls compare on frozen qualified inputs, with identical timing boundaries, per-input batches and a recorded environment?

RQ3: **Structure and implementation cost.** How do the actual operations support their source-level bounds, and what can finite timing and structural coverage reveal about their concrete costs?

The first question uses source-linked reasoning and finite validation, including independent models, checked audits and same-backend replay. The second uses predeclared matched measurement and independent archive validation. The third distinguishes amortized analysis from descriptive observations; neither timing trends nor historical counter associations supply a causal or asymptotic proof.

1.4. Contributions

1. An executable Step 1/2/3 reconstruction with explicit orientation, endpoint and first-point decisions, independently maintained output, and a shared loop across both backends.
2. A heterogeneous red–black finger-tree representation with stable pair/leaf identities, endpoint insertion, two-sided split, local repair and bounded header ownership updates, alongside the ordinary-list baseline.

3. A source-level manual argument relating reversed pointers to logical trees, paying mixed-operation costs with two potentials, and connecting the published locality lemma and a finite-parent supplement to actual calls.
4. A reproducible evidence workflow separating qualification, calibration, formal batches and historical results, retaining original outputs and failures, and reconstructing schedule and statistics with a separate validator.

Level-linked search trees are not implemented. The reviewed tree cost bound uses unit-cost comparisons and machine-word operations, expected dictionary lookup and amortized growth. It does not promise unconditional Python wall time or a practical speedup at the measured sizes.

1.5. Headline Findings

Both input sets passed v2 qualification, and all 66,560 formal sample calls returned correct output in 6,080 batches. The tree is slower on every measured input. In the common set, the median of within-input tree/list median ratios is 2.898, with range 1.114–3.535; in the targeted split set it is 3.180, with range 3.147–3.213. Related cases and mirrors are not independent random samples. No future crossover is inferred from the narrowing common-set gap.

The common inputs exercise no two-sided splits, despite their nesting strata. The targeted group supplies that path for smaller sides 3, 7 and 19, but does not represent all split scales or update histories. These observations explain why qualification, path coverage and practical performance must be reported separately. The ordinary backend's quadratic copying lower bound and the tree's reviewed linear total in the stated model are source arguments, not curve fits. Full results and limits appear in chapters 6 and 7.

1.6. Thesis Structure

Chapter 2 introduces the geometry and prior algorithms. Chapter 3 develops the executable incremental procedure, and chapter 4 its representations, checks and cost argument. Chapter 5 specifies input qualification, calibration and matched measurement. Chapter 6 presents current results with a separate historical summary; chapter 7 states their boundaries. Chapter 8 answers the research questions and identifies remaining work. The appendix binds claims and artifacts to their producing revisions.

2. Background

2.1. Jordan Sequences and the Sorting Problem

Let C be a simple plane curve that crosses the x-axis wherever it meets the axis. Listing the abscissas of the intersection points in the order in which they occur along C gives a sequence

$$z_1, z_2, \dots, z_n. \tag{2.1}$$

Such a realizable sequence is called a *Jordan sequence*. The Jordan sorting problem is to recover the order of the same points along the x-axis, that is, their increasing coordinate order, from their order along the curve (Hoffmann et al. 1986, p. 170). The 1990 presentation uses the term Jordan curve for both open and closed simple curves; the property needed here is the absence of self-intersections (Fung et al. 1990, pp. 85–86).

This formulation gives two distinct orders on one set of points:

Curve order is the input order $z_1, \dots, z_n$ obtained by traversing C.
Axis order is the output order obtained by moving from left to right along the x-axis.

The values are therefore not arbitrary keys presented in an arbitrary permutation. They carry a geometric realizability constraint inherited from the simplicity of C. The implementation studied in this thesis specializes this model to distinct, mutually comparable coordinate values, as stated in section 4.2.

The problem arose from geometric applications rather than from a desire to replace general-purpose sorting. The 1986 paper relates it to ordering the intersections of a simple polygon with a line and to ordering intersections of two Jordan curves. The 1990 paper develops polygon clipping as its main application and also notes the role of Jordan sorting in simple-polygon triangulation (Hoffmann et al. 1986, pp. 170–171; Fung et al. 1990, pp. 85–86). In these settings, intersection points can be obtained naturally in boundary or curve order, while the next geometric stage requires their order along another line or boundary.

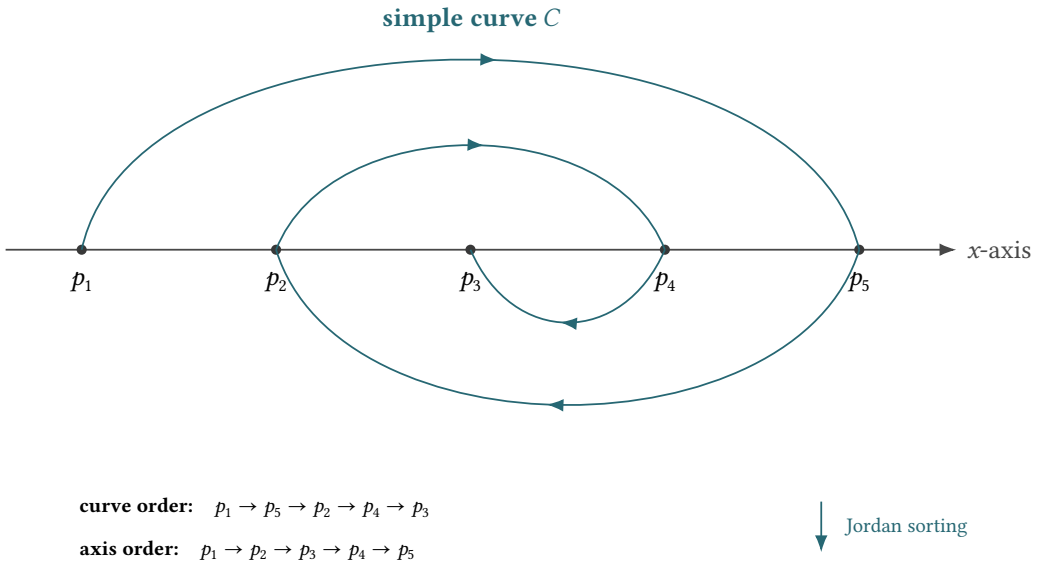


Figure 2.1.: Illustration of the Jordan sorting problem. The simple curve encounters the five intersections in curve order $(p_1, p_5, p_2, p_4, p_3)$, whereas their axis order is $(p_1, p_2, p_3, p_4, p_5)$. Jordan sorting recovers the axis order from the curve order.

2.2. The Structural Restriction Behind Jordan Sorting

Assume, after reflection about the x-axis if necessary, that the first arc lies above the axis. Consecutive intersections delimit alternating arcs above and below it. The pairs

$$\{z_1, z_2\}, \{z_3, z_4\}, \ldots \tag{2.2}$$

belong to the upper side, while

$$\{z_2, z_3\}, \{z_4, z_5\}, \ldots \tag{2.3}$$

belong to the lower side. Each pair determines the open interval between its two coordinates on the axis.

Two intervals from the same side cannot cross. If one endpoint of one such pair lies strictly inside another pair's interval, then its other endpoint must lie there as well; otherwise the corresponding arcs would intersect. Consequently, same-side intervals are disjoint or nested. Replacing the smaller endpoint by a left parenthesis and the larger by a right parenthesis therefore produces one properly nested parenthesis system for each side (Hoffmann et al. 1986, pp. 172–173).

The containment relation gives an ordered forest for each parity family. A pair's parent is its smallest same-family enclosing pair, and siblings are ordered by their left endpoints. Adding a dummy pair

spanning $(-\infty, +\infty)$ turns each forest into a rooted ordered tree. These are the upper and lower family trees used by both historical algorithms (Hoffmann et al. 1986, p. 173; Fung et al. 1990, p. 87). The full notation and executable pair model used in this thesis are developed in section 3.2; here the important point is that simplicity turns a geometric constraint into nested ordered structure.

This structure is what makes specialized algorithms possible for valid inputs. While processing points in curve order, the algorithms maintain both a sorted list of the processed coordinates and the two family trees. Inserting the next pair into its family identifies a local position for the new coordinate in the sorted list (Hoffmann et al. 1986, pp. 173–175). The remaining difficulty is not an unrestricted search through all processed values, but the efficient maintenance and splitting of ordered family lists.

2.3. The 1986 Linear-Time Algorithm

Hoffmann et al. (1986) gave an algorithm that both recognizes and sorts Jordan sequences in $O(n)$ time. It processes the sequence from left to right in curve order, maintaining a doubly linked sorted list and the upper and lower family trees. When a new interval would violate the required nesting, the recognition algorithm aborts; if all points are processed, the maintained list is the sorted result (Hoffmann et al. 1986, pp. 173–175).

The central data-structural operation is splitting a family list around the new interval. A circular doubly linked representation permits local insertion but can require a scan proportional to the smaller side of a split. Across a sequence of splits this yields an $O(n \log n)$ implementation, no better asymptotically than general sorting (Hoffmann et al. 1986, pp. 175–176). The paper obtains its linear bound by replacing those lists with circular level-linked $(2, 4)$ trees.

A level-linked $(2, 4)$ tree stores list items at the leaves of a balanced multiway search tree and connects nodes at equal height. These links support finger searches whose cost depends on the shorter circular distance between the starting item and the target. Together with tree splitting, concatenation, and an amortized analysis, the representation supports the family-list operations within $O(n)$ total time (Hoffmann et al. 1986, pp. 177–183). Thus the 1986 result couples two ideas:

1. reduce Jordan recognition and sorting to incremental family-list manipulation; and
2. realize that manipulation with a specialized level-linked data structure.

The second component is essential to the published linear-time bound.

2.4. The 1990 Simplification

Fung et al. (1990) retained the linear theoretical bound while simplifying both the primary structure and the operations required of it. A locality lemma shows that, after a pair has been added, the relevant children of a family remain in one or two contiguous sibling lists. As a result, the sorting algorithm needs only three abstract list operations:

1. create a singleton list;
2. insert an item at the front or back of a known list; and
3. split a list at a specified item or key.

In particular, the simplified formulation avoids the 1986 operation that extracts a middle section and concatenates the two remaining ends (Fung et al. 1990, pp. 86, 88–90).

The algorithm again processes intersections incrementally. It maintains a sorted list, upper and lower family trees, and sibling lists encoding the local family order. Neighboring processed coordinates locate the relevant same-family boundaries; insertion of the new pair and any necessary sibling split then determine where the new point enters the sorted list. The exact Step 1/2/3 reconstruction used in this thesis, including the symmetric orientation and endpoint clarifications, is the subject of chapter 3 and is not repeated here.

For the theoretical implementation, the 1990 paper proposes heterogeneous finger trees. It states constant amortized time for singleton-list creation and boundary insertion, and a logarithmic-in-the-smaller-side amortized bound for splitting. A potential argument charges the split cost across the operation sequence, giving $O(m)$ total time for m list operations and hence an $O(n)$ Jordan-sorting bound (Fung et al. 1990, pp. 90–91). This is a theorem about the algorithm together with that efficient list representation, not about every concrete implementation of the three-operation interface.

The 1990 paper also states that the sorting method extends to recognition and to the error-correcting Jordan-sorting task used in the polygon-triangulation algorithm of Tarjan and Van Wyk (1988), and that neither extension changes its $O(n)$ bound (Fung et al. 1990, p. 91). The error-correction task belongs to that separate triangulation work; neither it nor recognition is evaluated in this thesis.

2.5. Related Work and Positioning

2.5.1. Ordered Lists and Finger Search

The specialized list backends used for Jordan sorting belong to a broader line of work on ordered-list representations with fingers and locality-sensitive access. Brown and Tarjan (1978) study linear lists

with movable fingers, and Brown and Tarjan (1980) analyze a 2–3-tree representation for sorted lists whose costs reflect locality of reference. Huddleston and Mehlhorn (1982) develop level-linked weak B-trees for arbitrary sequences of sorted-list updates. The level-linked $(2, 4)$ trees used by the 1986 Jordan-sorting algorithm are part of this historical data-structure lineage, rather than a property of Jordan sequences alone.

These works provide context for the historical linear-time theorem. The ordinary-list backend retains explicit scan, copy, slice and rebinding costs; the heterogeneous backend instead follows the finger-tree approach and is analyzed against its actual source representation in chapter 4.

2.5.2. Jordan Sorting and Geometric Applications

The first symposium publication of the linear-time Jordan-sorting result appeared in 1985 (Hoffmann et al. 1985); the 1986 journal article used throughout this thesis gives the expanded algorithm and data-structure account. The later simplification connected Jordan sorting to polygon clipping (Fung et al. 1990), while the error-correcting task appears in the polygon-triangulation algorithm of Tarjan and Van Wyk (1988). Subsequent work by Bar-Yehuda and Chazelle (1994) used a generalized form of Jordan sorting together with point location and interval trees to triangulate collections of disjoint Jordan polygonal chains.

Other work pursued alternatives to the 1986 and 1990 dynamic-list algorithms. Aurenhammer (1987) related Jordan sorting to convex-hull computation for certain non-simple polygons. Sojka (1997) instead emphasized simpler algorithms intended to be practical at moderate input sizes, including an $O(n \log n)$ worst-case method and an $O(n \log \log n)$ worst-case method. A later average-case study analyzed a new simple algorithm under an explicit random-Jordan-sequence model, identified conditions for linear expected time, and supplemented the analysis with simulation (Sojka 1999). These are distinct algorithmic and analytical directions. This thesis selects the 1990 procedure as an object of executable reconstruction; it neither claims that procedure is the only post-1986 approach nor implements or compares the alternatives in this paragraph.

This literature shows that Jordan sorting participates in a wider computational-geometry research line, rather than being only an isolated sorting problem. The present thesis nevertheless has a narrower target: it reconstructs and evaluates valid-input sorting. It does not implement a new theoretical list structure, polygon clipping, triangulation, error correction, or recognition.

2.6. Position and Scope of This Thesis

This thesis separates three layers that are easy to conflate:

Geometric validity. A candidate sequence is checked against the Jordan-validity model used for certification before the paper core is timed.

Valid-input sorting. The reconstructed paper control flow maintains its own partial order and recovers the final output from that state.

List representation. Sibling lists use ordinary Python lists or the implemented heterogeneous finger-tree backend, with separate cost analyses.

Accordingly, the empirical study concerns sorting of oracle-certified valid inputs. Recognition remains separate, as specified in section 5.1. The paper core does not consume oracle-sorted output, whereas the simplified reference pipeline deliberately remains oracle-backed; the implementation boundary is detailed in section 4.4.

The ordinary-list representation supports an executable and auditable reconstruction, but it may scan, slice, copy, and rebind list contents. This thesis does not implement the level-linked trees of 1986. Its separate heterogeneous backend requires source-level representation, locality and amortized-cost arguments; merely implementing the interface would not transfer the papers' linear-time theorem. The consequences for empirical interpretation are stated in sections 7.2 and 7.4.

3. Algorithm Reconstruction

3.1. Purpose and Reconstruction Boundary

The Jordan sorting problem starts with the intersections of a simple curve and the x-axis in curve order,

$$z_1, z_2, \dots, z_n, \tag{3.1}$$

and asks for the same points in increasing x-coordinate order. The 1990 algorithm of Fung et al. (1990) processes this sequence incrementally while maintaining a sorted list and two ordered family trees. Its presentation is compact: the increasing case is described explicitly, the decreasing case is called symmetric, and some endpoint details are left implicit. This chapter states the executable reconstruction used in the thesis.

Three kinds of statements must therefore remain distinct:

Explicit source rule. A definition or step is stated directly in the 1990 paper, such as the upper/lower family trees, predecessor and successor boundary searches, and increasing Step 3.

Symmetric reconstruction. A decreasing rule is obtained by reflecting the explicit increasing rule: predecessor and successor roles, left and right split sides, child extremes, and insertion direction are exchanged.

Executable clarification. A detail is fixed so that the compact paper description becomes unambiguous code. The geometric child endpoint and the odd-index z_1 output-anchor rule fall into this category.

The last category is not presented as the unique intended wording of an unstated paper detail. It is an implementation-level interpretation supported by the related 1986 algorithm of Hoffmann et al. (1986), by reflected counterexamples, and by bounded exhaustive validation over oracle-certified inputs. The validation evidence checks this reconstruction; it does not replace a topological proof of the source algorithm.

The chapter also separates algorithmic control flow from data-structure complexity. The 1990 paper obtains its linear bound with heterogeneous finger trees. Both thesis backends realize the same high-level operations. Ordinary Python lists incur the quadratic copying lower bound in section 4.6;

the real finger-tree backend has a separate source-linked representation and amortized-cost argument there.

3.2. Pairs, Intervals, and the Two Families

Input values are assumed to be distinct and mutually comparable. For each $i \geq 2$, define the curve-order pair

$$P_i = \{z_{i-1}, z_i\}. \tag{3.2}$$

The two stored endpoints retain curve order. Their geometric interval is described separately by

$$\ell_i = \min\{z_{i-1}, z_i\}, \qquad r_i = \max\{z_{i-1}, z_i\}. \tag{3.3}$$

A pair P_i encloses a finite point x when

$$\ell_i < x < r_i, \tag{3.4}$$

and it encloses another pair P_j when

$$\ell_i < \ell_j < r_j < r_i. \tag{3.5}$$

After reflecting the curve about the x-axis if necessary, the pairs with even end index belong to the upper family and those with odd end index belong to the lower family:

$$P_2, P_4, P_6, \ldots \in \mathscr{F}^+, \qquad P_3, P_5, P_7, \ldots \in \mathscr{F}^-. \tag{3.6}$$

For pairs in one family, simplicity of the curve implies that their intervals are disjoint or nested. The enclosure relation therefore forms a forest. Each set of siblings is ordered from left to right by increasing ℓ_i.

The reconstruction adds a distinct dummy pair to each family. Each dummy conceptually spans $(-\infty, +\infty)$ and becomes the parent of every pair that has no finite enclosing parent. The two forests are thereby represented as rooted ordered trees, but the upper and lower dummy identities remain separate.

3.2.1. Families and Sibling Lists

Following the locality argument of Fung et al. (1990), the children associated with a family can be represented by one or two ordered sibling lists. The most recently added pair divides the family

region into a left and a right part; children on either side remain contiguous in sibling order. This locality is what reduces the update to three abstract list operations:

1. create a singleton sibling list;
2. insert a pair at a known list boundary; and
3. split a sibling list at the value of the new point.

The algorithm never requires arbitrary insertion into the middle of a sibling list. A split may produce an empty side, but an empty side is not retained as a live sibling list.

3.3. Maintained State and Prefix Invariant

After processing $z_1, \dots, z_k$, the conceptual state contains:

- a partial-order list S_k containing exactly those k points in increasing value order;
- every finite pair $P_2, \dots, P_k$ in its parity family;
- one parent and one sibling-list owner for every finite pair; and
- the two family-specific dummy roots.

The central prefix invariant is

$$S_k = \mathrm{sort}(z_1, \dots, z_k). \tag{3.7}$$

Here `sort` is mathematical notation for the required order, not an operation invoked by the paper core. The maintained list is bounded by identity-based negative- and positive-infinity sentinels so that predecessor and successor queries are defined at both ends.

The family-tree invariant and equation (3.7) are updated together. Pair insertion locates the interval in which z_i belongs, sibling splitting transfers any newly enclosed children, and the resulting family boundary determines the local insertion anchor in S_{i-1}.

3.4. Initialization

Lengths zero, one, and two are handled directly by fixed-size comparison logic. For $n \geq 3$, the incremental state is initialized by ordering z_1, z_2, z_3 with fixed-size comparison logic and creating

$$P_2 = \{z_1, z_2\} \qquad \text{in the upper family,} \tag{3.8}$$

$$P_3 = \{z_2, z_3\} \qquad \text{in the lower family.} \tag{3.9}$$

The upper dummy owns a singleton sibling list containing P_2, and the lower dummy owns one containing P_3. Thus S_3 satisfies equation (3.7), both parity families have a root, and every finite pair has exactly one owner before the main loop begins.

3.5. Boundary Selection

Iteration $i \geq 4$ creates P_i in the family determined by the parity of i. Steps 1 and 2 inspect the immediate neighbors of z_{i-1} in S_{i-1}.

3.5.1. Step 1: Predecessor-Side Boundary

Let v be the predecessor of z_{i-1}. If i is odd and $v = z_1$, replace v by the predecessor of z_1. If the resulting neighbor is the negative sentinel, let A_i be the dummy pair of the current family. Otherwise, let A_i be the already processed same-family pair containing v.

3.5.2. Step 2: Successor-Side Boundary

Let w be the successor of z_{i-1}. If i is odd and $w = z_1$, replace w by the successor of z_1. If the resulting neighbor is the positive sentinel, let B_i be the current family dummy. Otherwise, let B_i be the processed same-family pair containing w.

A processed point can belong to both an upper and a lower pair. The parity of i selects the relevant incident pair without using a global rank map. The special treatment of z_1 is necessary because z_1 belongs to P_2 but to no lower-family pair. These two neighbor adjustments select family-tree boundaries; they are distinct from the output-anchor adjustment introduced in section 3.8.2.

3.6. Step 3: Increasing Orientation

Assume first that $z_{i-1} < z_i$. Let $N = P_i$ denote the new pair. The predecessor-side pair A_i controls where N enters the family tree, while the successor-side pair B_i controls whether an existing sibling list must be split.

3.6.1. Step 3(a): Insert the New Pair

If A_i encloses z_{i-1}, then N becomes its child in a new singleton sibling list. Otherwise, A_i is the last item of its sibling list; N is inserted immediately after A_i and inherits the same parent. In either case, N obtains exactly one parent and one sibling-list owner.

3.6.2. Step 3(b): Split Off Newly Enclosed Children

If B_i encloses z_{i-1}, N acquires no child list. Otherwise, B_i is the first item of an existing sibling list L. Splitting L at z_i produces a left side and a right side. For a valid input, no pair may straddle the split value: both endpoints of each child lie on one side. The left side, whose pairs are newly enclosed by N, becomes a child list of N; the right side remains with the previous owner of L.

3.6.3. Step 3(c): Insert the New Point

If N acquired no children, the base output anchor is z_{i-1}. Otherwise, let C be the rightmost child pair of N and choose the geometric right endpoint $r(C)$ as the base anchor. After the possible odd-index adjustment in section 3.8.2, insert z_i immediately after the output anchor.

3.7. Step 3: Decreasing Orientation

For $z_i < z_{i-1}$, the reconstruction uses the order reflection of the increasing case. This is the executable meaning assigned to the 1990 paper's statement that the other case is symmetric.

Step 3(a) now uses B_i. If B_i encloses z_{i-1}, N becomes its child in a singleton sibling list. Otherwise, B_i is the first item of its list; N is inserted immediately before it and inherits its parent.

Step 3(b) uses A_i. If A_i encloses z_{i-1}, no split is needed. Otherwise, A_i is the last item of its sibling list. The list is split at z_i; the right side is transferred to N, while the left side remains with the old owner.

For Step 3(c), the no-child base anchor is again z_{i-1}. With children, let C be the leftmost child pair of N and use its geometric left endpoint $\ell(C)$. After the reflected odd-index adjustment, insert z_i immediately before the output anchor.

The complete reflection is summarized in table 3.1.

Table 3.1.: Increasing and decreasing Step 3 roles in the executable reconstruction.

Operation	Increasing	Decreasing
Step 3(a) boundary	predecessor-side A_i	successor-side B_i
Boundary insertion	after last sibling	before first sibling
Step 3(b) boundary	successor-side B_i	predecessor-side A_i
Acquired split side	left	right
Child extreme	rightmost child	leftmost child
Geometric anchor	right endpoint	left endpoint
Output insertion	after anchor	before anchor

3.8. Executable Anchor Clarifications

The point insertion in Step 3(c) is where the compact paper notation requires the most care. Two independent distinctions are necessary.

3.8.1. Curve Endpoint Versus Geometric Endpoint

The 1990 increasing step refers to the rightmost child $\{z_{m-1}, z_m\}$ and inserts the new point after z_m. A pair record, however, stores its endpoints in curve order, and its second curve endpoint is not necessarily its geometric right endpoint. Reading z_m mechanically as the second stored endpoint fails on valid reflected orientations.

For example, consider

$$(3, 2, 1, 4). \tag{3.10}$$

At $i = 4$, the acquired child is $P_2 = \{3, 2\}$. Inserting 4 after the second curve endpoint 2 would yield $(1, 2, 4, 3)$. Inserting after the child's geometric right endpoint 3 yields the correct order. The decreasing counterpart

$$(2, 3, 4, 1) \tag{3.11}$$

similarly requires insertion before the child's geometric left endpoint 2, not before its second curve endpoint 3.

The reconstruction therefore uses $r(C)$ for the increasing branch and $\ell(C)$ for the decreasing branch. This choice agrees with the earlier family-list account of Hoffmann et al. (1986), in which both endpoints occur in sorted family lists and the new point is inserted next to the located boundary item. It is nevertheless labeled an executable clarification rather than attributed to a fully explicit sentence in the 1990 presentation.

3.8.2. Odd-Index z_1 Output Anchor

Steps 1 and 2 compensate for the fact that z_1 has no lower-family pair. That boundary-pair correction does not cover every final output insertion. The 1986 description records a separate anomaly: on an odd iteration, if z_1 lies between the ordinary family-list boundary and the new point, the new point is inserted next to z_1 (Hoffmann et al. 1986, p. 175).

The executable increasing rule is

$$\text{if } i \text{ is odd and } a_i < z_1 < z_i, \quad a_i \leftarrow z_1, \tag{3.12}$$

where a_i is the geometric base anchor. The decreasing reflection is

$$\text{if } i \text{ is odd and } z_i < z_1 < a_i, \quad a_i \leftarrow z_1. \tag{3.13}$$

For the decreasing sequence prefix

$$(1, 2, 3, 4, 6, 7, 0), \tag{3.14}$$

the ordinary geometric base anchor at $i = 7$ is 2. Inserting 0 before 2 would leave 1 on the wrong side. Because $0 < z_1 = 1 < 2$, the output anchor is changed to z_1, and insertion before it gives the correct prefix order. The reflected sequence $(6, 5, 4, 3, 1, 0, 7)$ exercises equation (3.12). These cases demonstrate why boundary selection and output-anchor correction must be modeled as separate stages.

3.9. Ownership Transfer and Failure Semantics

A sibling split changes several linked facts at once: the old list ceases to exist, up to two nonempty output lists are created, the retained side stays with the old parent, the acquired side moves under N, and every moved pair must name its new list and parent. Conservation and unique ownership are required after a successful transition.

The ordinary backend first forms a split plan without changing live state; its commit checks the plan, parent and partition before updating ownership. The tree instead uses a planned transfer source, path-local splitting and header adoption, without materializing that list-based plan. Both must restore the logical ownership conditions before completing the iteration. In the revised implementation, an update or postcondition error terminates sorting. Updates already performed are not rolled back, and the failed state must be discarded. No partially updated state is accepted as a completed iteration or returned as a sorting result.

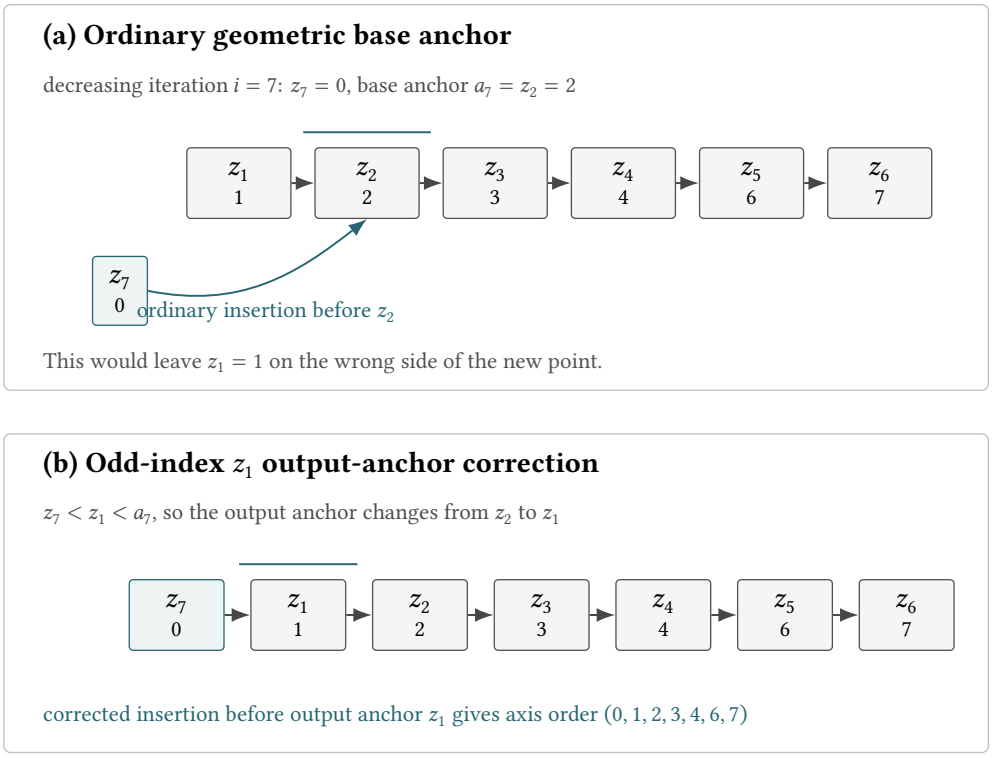


Figure 3.1.: Local output-anchor selection in decreasing Step 3(c) for the prefix $(1, 2, 3, 4, 6, 7, 0)$. The ordinary geometric base anchor is z_2. Because the odd-index condition $z_7 < z_1 < z_2$ holds, the output anchor is corrected to z_1 before z_7 is inserted.

The earlier implementation used rollback on these error paths. That recovery mechanism is distinct from the algorithmic requirement to preserve ownership on successful updates. The two versions and their experimental scope are distinguished in section 4.1; the concrete update is described in section 4.3.3.

3.10. End-to-End Procedure

The full valid-input sorting procedure is:

1. Materialize the input once and handle lengths zero, one, and two directly.
2. For $n \geq 3$, initialize S_3 by fixed-size ordering of the first three points, then create P_2, P_3, the two dummies, and their singleton sibling lists.
3. For each $i = 4, \ldots, n$:

   Step 1. Compute predecessor-side boundary A_i.

   Step 2. Compute successor-side boundary B_i.

   Orient. Choose increasing or decreasing Step 3 from the comparison of z_{i-1} and z_i.

   Update. Insert P_i, split and transfer any enclosed children, and insert z_i using the resulting geometric output anchor.

   Commit. Advance the processed prefix only after the output insertion succeeds.
4. Return the values obtained by traversing S_n.

At the end of each successful iteration, the required online conditions are:

- S_i contains each processed point exactly once and is strictly increasing;
- every processed finite pair exists in the family dictated by its parity;
- every finite pair belongs to exactly one live sibling list and names that list's owner as its parent;
- sibling order follows geometric left endpoints, parent chains are acyclic, and each chain reaches the appropriate dummy root; and
- split outputs are disjoint and preserve the input pair set.

The final return value is thus recovered from the maintained order S_n. Oracle certification may establish the valid-input precondition before this procedure is called, but oracle-sorted output is not an input to these steps. The implementation-level certification boundary is detailed in section 4.4.

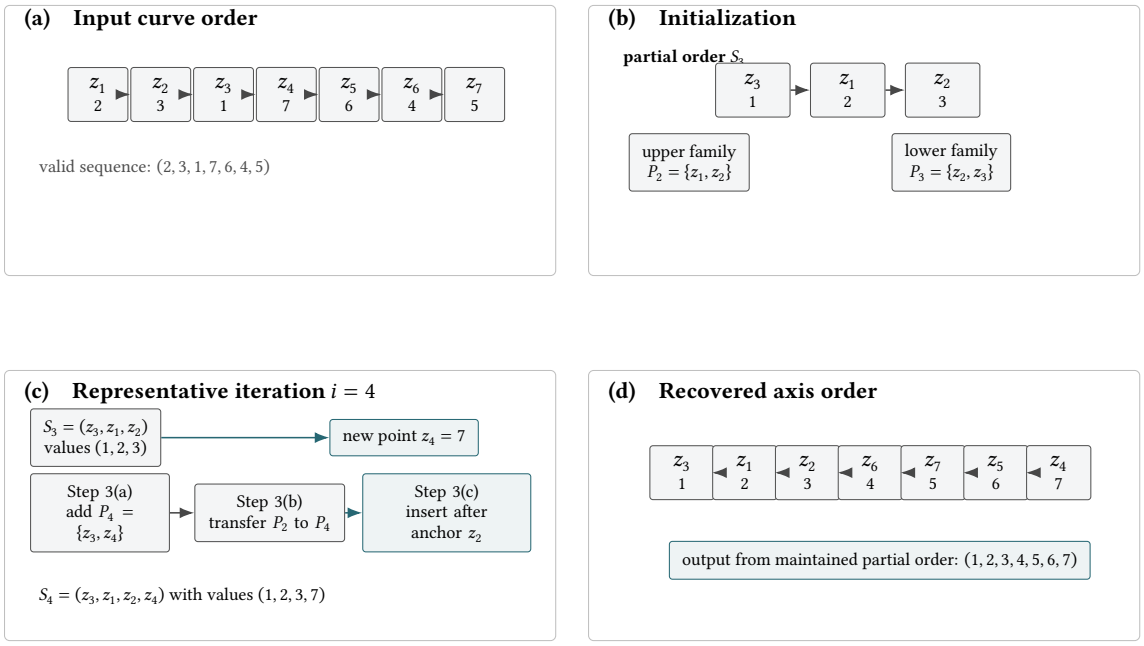


Figure 3.2.: Running example on the valid sequence $(2, 3, 1, 7, 6, 4, 5)$. The panels show the input, initialization, the representative update at $i = 4$, and the recovered axis order. At $i = 4$, P_4 acquires P_2 in Step 3(b), then Step 3(c) inserts z_4 after the geometric anchor z_2.

3.11. Correctness and Complexity Boundaries

The source-level correctness argument rests on the nesting property of same-family pairs and the locality lemma: the boundary pairs found beside z_{i-1} identify where P_i belongs, and newly enclosed children form a contiguous split side. The executable reconstruction turns those facts into the prefix and ownership invariants above. Focused counterexamples fix the ambiguous anchors; mirrored tests check the decreasing reconstruction; and full state diagnostics compare the maintained order and family forest with a deterministic replay. These are implementation correctness controls, not an independent proof that every unspecified detail has only one possible interpretation.

The theoretical papers and the thesis implementation have different backend claims. Hoffmann et al. (1986) use level-linked search trees, while Fung et al. (1990) reduce the required operations to singleton-list creation, boundary insertion, and splitting, then implement them with heterogeneous finger trees. The ordinary backend preserves those abstract operations using Python lists. A sibling split scans and materializes list contents, both insertion boundaries copy the current list, and ownership rebinding can depend on list size. Membership and ancestor checks can also scan lists in its minimal mode. The tree backend instead uses the physical operations described in chapter 4; its source-level bound and the ordinary backend's costs must not be interchanged.

4. Implementation

4.1. Implementation Scope and Versions

The code separates an oracle-backed reference pipeline from the shared-core reconstruction of Fung et al. (1990). The reference function `simplified_jordan_sort` certifies the input, constructs family trees, statistics and a trace, and returns the oracle's sorted output. In contrast, `paper_jordan_sort_valid` executes the reconstructed algorithm and returns `state.partial_order.to_list()`. Its core does not use the oracle's output or a global rank map. It assumes a pre-certified valid Jordan sequence with distinct, mutually comparable values.

Both current backends propagate errors without rollback. Current matched timings in chapter 6 measure these programs; an explicitly separate section retains the earlier rollback-enabled study. Neither comparison establishes a causal benefit of removing recovery work. Source bindings remain in section A.2.

Chapter 3 specifies initialization, both orientations of Step 1/2/3, and executable anchor clarifications. Here the focus is state representation and maintenance. The real tree shares the sorting loop and has bounded correctness evidence and a source-level manual cost/locality review. Qualification remains distinct from that mathematical argument.

4.2. Data Representation

An immutable `PointRef(paper_index, value)` separates one-based paper identity from a point's comparable value. A `PairRecord` retains curve endpoint IDs and family membership; geometric endpoints are obtained by comparing their values. The distinction and its consequences for output anchors are specified in section 3.8.1.

`PaperJordanState` holds these records, the processed-prefix length, the partial order, the end-index-to-pair mapping, the sibling backend and two family roots. It also holds a fixed execution policy, optional trace and counters, and mandatory stage results. Sorting and diagnostics use the same runner and mirrored orientation branches.

4.2.1. Partial Sorted Order

`SortedOrderList` is doubly linked, with identity-based infinity sentinels and a point-ID-to-node mapping. Neighbor access and insertion use local links. Before mutation, insertion checks that the value lies strictly between its adjacent values. Final traversal excludes the sentinels and returns the original values. Mapping access uses Python dictionaries; its cost must not be silently treated as a deterministic worst-case constant.

4.2.2. Sibling Lists and Ownership

Each `SiblingList` stores ordered finite pair IDs and one owner. A pair caches its parent and containing-list IDs, together with its ordered child-list IDs. Every owned finite pair occurs in exactly one live list, whose owner equals its parent. A parent has at most two ordered child lists; finite parent chains must reach the same-family dummy without cycles. The family dummies are roots, not finite list items.

The ordinary backend retains this eager representation. Since commit `71e3e71`, the shared core uses boundary-list, owner and extreme-child access interfaces instead of reading those fields directly. General pair-liveness checks do not require the pair to be a list endpoint.

The tree backend stores stable pair handles in external leaves, with internal separator nodes, two endpoint fingers and a list header. Outer rib pointers point toward the root; interior pointers point toward children. Separators store the maximum left key. Rank is black height excluding external black leaves. For internal nodes, span counts all leaves in the INTERIOR role, the inward branch on a rib, and zero in the ROOT role. External leaves always have span one, including a singleton root. The header separately stores total size. Ownership belongs to the header and at most two parent child slots, avoiding per-leaf rebinding. Boundary slots only identify designated endpoints; they are not arbitrary-owner lookup caches.

4.3. Core–Backend Interface and Update Semantics

4.3.1. Initialization, Boundaries and Stage State

The initialization in section 3.4 materializes all point records but orders only the first three values; shorter inputs use fixed-size comparisons. Subsequent iterations use the predecessor and successor boundaries defined in section 3.5. The backend handles singleton creation, boundary insertion, splitting and ownership transfer; the partial-order structure handles point insertion.

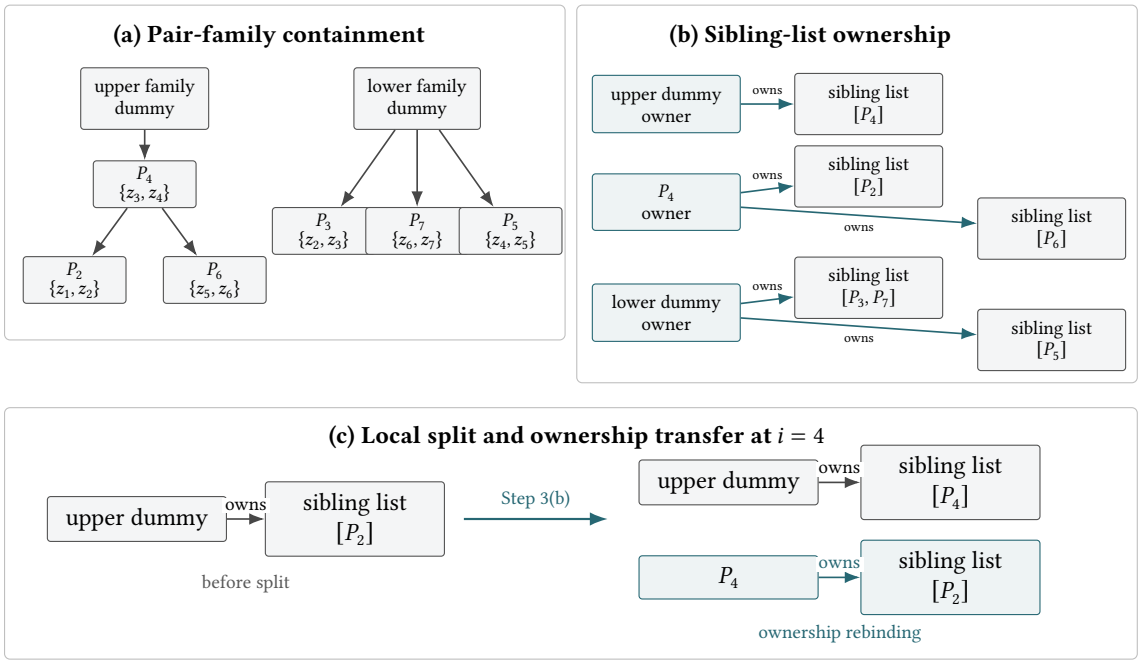


Figure 4.1.: Pair families and sibling-list ownership in the example of figure 3.2. Step 3(b) transfers P_2 from a list owned by the upper dummy to a child list of P_4.

Boundary selections and Step 3 results are stored even with trace disabled. Later stages consult them to reject incorrect order, repeated stages and inconsistent parameters. They are execution state, not merely optional logging. Optional backend-work observations are stored separately from these mandatory results; complete logical snapshots belong to the audit path.

4.3.2. Pair Insertion and Failure Propagation

For the insertion rules in sections 3.6.1 and 3.7, the core checks the boundary, registers the new pair and calls singleton creation or boundary insertion. A later error may leave that registration in place. The exception propagates; the state is discarded rather than restored or reused. This contract applies to both orientations.

4.3.3. Split Plans and Ownership Transfer

The ordinary backend implements the partition specified in sections 3.6.2 and 3.7 using an immutable `SplitPlan`. It records the original sequence and both ordered partitions before mutation. The plan sup-

ports stale-plan rejection, order checks and pair-set conservation; these functions remain necessary without rollback.

Commit checks the plan and ownership preconditions, prepares nonempty output lists and affected child slots, retires the input list, publishes the new lists, and rebinds the pairs. Touched-state postconditions are checked before success is returned. Rejection covers stale plans, straddling pairs, ownership mismatches, cycles, unowned parents and a third child list. Errors during mutation or postcondition checks propagate without restoring earlier state. Output-list preparation is ordinary update work, not a recovery backup.

The tree starts boundary insertion at a finger and repairs locally. A split with both sides nonempty races from both ends, detaches a component, splits its downward representation and joins by black height. Path sizes and header totals supply output sizes without enumerating leaves. Every successful split, including a one-sided result, retires its source ID and allocates fresh IDs for nonempty outputs; boundary insertion preserves its live ID. Pair handles, leaves and existing descendant groups survive transfer. Before Step 3(a), the core identifies the other group's transfer source without splitting it. Step 3(b) consumes that exact source; pending split/transfer state must clear before Step 3(c) inserts the output point.

4.3.4. Point Insertion and Completion

The backend exposes the relevant extreme child; the core selects its geometric endpoint and applies the separate odd-index z_1 anchor rule of section 3.8.2. These executable clarifications and their counterexamples remain in section 3.8; they are not attributed to fully explicit wording in the 1990 paper. The partial order performs the resulting adjacent insertion. Only after it succeeds does Step 3(c) advance `processed_count`.

4.4. Certification, Diagnostics and Replay

The wrapper `certified_paper_jordan_sort` runs the oracle and rejects invalid candidates before entering the pure sorter. Certification is outside the minimal core's timing scope. An internal exception is propagated by both sorting and diagnostic entry points: neither retries, returns a partial output, nor falls back to the oracle or reference pipeline. Output recovery is reached only after the runner succeeds.

`paper_jordan_diagnostics_valid` selects `checked` and audits initialization and every completed prefix. It checks point identity and values, partial-order links and order, exact pair membership, ownership

and parent chains, stage records, trace and metrics. Deterministic replay reconstructs a fresh state using the same core, backend type and immutable configuration. It compares backend identity, partial order, registries, stage results, trace, work observations, metrics, and logical and physical snapshots. Unavailable backends fail explicitly rather than reverting to ordinary lists. This is same-core state-consistency evidence, not a second independent algorithm or a replacement for the external sorted-output check.

Qualification v2 instead advances two independently initialized checked instances through the same pausable core, audits and compares every completed prefix, then performs one final fresh checked replay. A separate fresh minimal call checks the measured configuration. It avoids reconstructing every prefix from scratch; this changes replay coverage and is explicitly versioned. Snapshots reuse one audit within a validation call, never a cross-call "already checked" cache. Old stage and trace records remain checked.

Failure-injection tests separately check propagation from pair insertion and split postconditions, including checks retained in `minimal`. Public-call tests require one split attempt and no output recovery after failure. They impose no rollback or failed-state reuse requirement.

4.5. Execution Policies and Observation Costs

Five immutable policies control trace recording, selected operation counters and complete backend validation after split commits. `checked` enables all three; `minimal` disables them. Intermediate policies isolate observation costs. All use the same Step 1/2/3 sequence and retain stage results, necessary local checks, input materialization and output recovery. The revised implementation has no rollback under any policy.

The tree's detailed work ledger is a separate configuration switch: minimal alone does not disable it. Qualification and calibration explicitly select `detailed_work=False`, suppressing field accumulation, ledger snapshots and per-operation reports. Local geometry and pending checks remain; access wrappers and switch checks still have runtime cost.

The archived studies instead timed the rollback-enabled implementation, including recovery-state preparation in `minimal`; a successful call need not actually roll back to incur that preparation. The historical scope is specified in section 5.3.3.

For each successful split, the ordinary backend reports an increment equal to the input-list length for both `split_items_scanned` and `split_items_copied`. They are logical split-volume measures, not exact interpreter-level access counts. They omit boundary-insertion copies, membership scans and repeated validation/materialization passes. Retained historical counter names do not establish

that splits dominate runtime. Logical volumes, directly observed work and wall-clock timings are therefore distinct forms of evidence. The shared core accumulates backend-reported work under an identified metric schema; it does not infer tree copying from the input-list length.

4.6. Actual Costs and Complexity Boundary

In the ordinary backend, either boundary insertion constructs a new Python list containing the existing IDs and the new pair. Rear insertion is not an in-place `list.append()`. Both orientations copy work proportional to the current sibling-list length. Split additionally scans and partitions the list, materializes its outputs and rebinds pair ownership.

"Local" describes the objects a safety check concerns, not constant cost. The ordinary backend's liveness check uses `pair_ids.count(pair_id)` for membership, and backend parent checks scan the containing list at each visited ancestor. For an ancestor chain A, this contributes $O(\sum_{a \in A} |L(a)|)$, where $L(a)$ contains a, rather than a constant per ancestor. These costs remain in `minimal`; complete checked validation and prefix replay add separate untimed work.

The valid increasing input $(1, 2, \ldots, 2m)$, $m \geq 2$, exposes a quadratic copying lower bound even without split. Starting from one pair per family, boundary insertions grow the sibling lists to m and $m - 1$ while Step 3(b) skips splitting. The number of existing pair IDs copied is

$$\sum_{j=1}^{m-1} j + \sum_{j=1}^{m-2} j = (m-1)^2. \tag{4.1}$$

Thus this input family incurs $\Omega(n^2)$ copying, including in `minimal`. This is a source-derived bound on one component of work, not a curve fit or a tight total bound for all inputs.

The ordinary backend instantiates neither the level-linked structure of Hoffmann et al. (1986) nor the heterogeneous finger-search structure of Fung et al. (1990). The implemented tree has now undergone a source-level manual review of representation preservation, mixed costs and locality, indexed in section A.2. This is a mathematical argument checked against the code, not machine verification or an independent external proof certification. Its main steps are as follows.

Decoding the reversed ribs gives an ordinary external red–black tree. Each rotation preserves the ordered leaf blocks and their separators; role changes update the affected spans and top caches. During deletion, a virtual black layer at the deficient component explains the temporarily unequal actual ranks. Propagation moves that layer upward; absorption or a terminal rotation restores equal actual ranks. Header retirement changes list identity without replacing existing leaves or their child groups.

Downward join also has a recursive transient: a red root may return with one red child on the updated side. Equal black heights and metadata still hold; the original black parent repairs that edge on return. Black-root repair returns an internally valid subtree, possibly with a red root. Since the join inputs are first blackened, its top-level return is internally valid before final root blackening. That last step alone cannot repair an internal red–red edge.

Let v be rank after root blackening. A downward join of heights a, b returns height c with $\max(a,b) \le c \le \max(a,b)+1$ and uses at most $2|a-b|+3$ descent steps. In split backtracking, the accumulated height a exceeds the untouched sibling height b by at most one. Thus each join costs at most $2(c-a)+5$ such steps; the two output chains telescope to $O(r+1)$ for selected component rank r. The finger race bounds r by $O(\log(s+1))$, where s is the smaller output size. This uses the component *before* an upward step as the wholly one-sided component; its parent may cross the cut. Empty-side splits take the endpoint fast path.

For logical internal nodes take

$$\Phi = \#\{\text{black nodes with two black children}\} + 2\#\{\text{black nodes with two red children}\}.$$

An insertion recoloring propagation decreases this potential by one; a continuing deletion propagation decreases it by two. Terminal repairs affect only constantly many weights. An opposite-rib walk occurs at most twice near the root and costs $O(r+D+1)$ for D deletion propagations. Downward joins and component boundaries add only $O(r+1)$ weights. One sufficiently large multiple of the same potential therefore pays for mixed propagation, including after split: boundary insertion is amortized $O(1)$ and split is $O(\log(s+2))$. The potential is never reset between operations.

For list sizes, $\Psi = c\sum_{L\neq\emptyset}(|L| - \log_2|L|)$ increases by at most a constant on creation/insertion. A two-sided nonempty split $m = a+b$, $1 \le a \le b$, changes it by $c\log_2((a+b)/(ab)) \le c(1-\log_2 a)$, paying the smaller-side term. An empty-side split leaves this potential unchanged, $\Delta\Psi = 0$. Locality supplies legal endpoint calls: the published latest-child re-entry lemma and a separate finite-parent crosscut argument imply the exact group boundary. A new pair takes a consecutive group portion; adoption restores the grouping, and the two odd-stage z_1 corrections restore the output neighbors. The full induction and root/nonroot mirror cases are in the review record.

Starting from constant-size initialization, these two nonnegative potentials and the constant work per shared-core stage give $O(n)$ expected/amortized source-level work under unit-cost comparisons and machine-word operations, with expected lookup and amortized dictionary growth. This does not bound arbitrary Python comparison methods, checked validation, or unconditional wall time, and does not predict a practical speed advantage.

5. Experimental Method

5.1. Study Scope and Version Boundary

Experimental algorithmics studies concrete programs, inputs and machines (McGeoch 2012); algorithm engineering separates implementation, analysis and evaluation (Sanders 2009). Accordingly, the current matched protocol and the historical measurements are distinct studies. The current ordinary-list and tree implementations have completed qualification, separate batch calibration and formal paired measurement. Current results in chapter 6 use source `4a83fa6`; the older rollback-enabled ordinary-list study remains a separate historical section. Neither calibration nor a later checker changes an old execution's source identity.

5.2. Current Inputs and Qualification

The frozen common set contains 140 unique rank inputs at ten lengths: 32, 33, 64, 65, 128, 129, 256, 257 and 512, 513, with two strict-flat and four each of low, medium and heavy nesting per length. Fixed constructors, seeds, candidate budgets, rank deduplication and first-eligible retention preceded timing. These are structural constructions, not random independent observations. All 140 cases pass both backends' v2 checks. Their 3,788 recorded splits per backend all have an empty output side: they exercise ownership transfer but not the tree's interior two-sided split path.

A separate targeted set uses the existing construction at $n = 128, 129$, $k = 3, 7, 19$, with each direction and its mirror: twelve cases, all qualified. Actual traces contain twelve two-sided splits per backend, with smaller sides 3, 7 and 19 occurring four times each. Each source has size 61 and a finite owner; later direct updates affect only the retained side. This bounded coverage does not represent all scales, ratios or follow-up operations. The two sets remain separate, not 152 independent random samples.

For each input/backend, v2 advances two independent checked instances, compares every completed prefix, and executes a third fresh checked replay at the end. Another fresh minimal call uses the intended timing configuration. Each output is compared to external sorting, without feeding that expectation into the core. These checks establish finite execution evidence, not a general proof.

The old fresh-replay-at-every-prefix protocol remains a bounded reference and is not described as identical coverage.

5.2.1. Historical Input Protocol

The archived formal study instead used 60 cases: at each $n \in \{32, 64, 128, 256, 512\}$, one flat, one nested and ten seeded incremental cases. Generation seed `20261201` and case identities determine the inputs. Duplicate hashes were rejected within family–size groups, not by a global rank rule. All 60 passed oracle validity, distinctness, structural profiling and checked ordinary-list diagnostics before warm-up. Failures stopped the run; checked outputs were compared externally. The ten-to-one family imbalance remains visible in historical aggregate results.

5.3. Compared Implementations and Call Scopes

The current comparison uses the same public Jordan core with ordinary-list or heterogeneous-finger storage, both minimal; the tree explicitly sets `detailed_work=False`. Every call initializes fresh state and recovers its output. Production source hashes match the qualified versions. Source or configuration changes require checking which qualification and measurements must be renewed. A tool-only archive check is recorded separately.

5.3.1. Python Sort

In the historical three-way comparison, `python_sort` sorts directly, without Jordan recognition or structural analysis.

5.3.2. Reference Pipeline

`simplified_jordan_reference` includes oracle validation, state and family construction, statistics, trace and result assembly. It is a pipeline, not a sorting-only primitive.

5.3.3. Historical Ordinary-List Core

`simplified_jordan_paper_ordinary_list` times the pre-certified rollback-enabled minimal core, including local checks, mandatory stage state, recovery preparation and output recovery. Successful calls still pay recovery preparation costs. Its ratios to the reference retain this difference in scope; they do not isolate the benefit of rollback removal or of tree storage.

5.4. Timing, Calibration and Scheduling

Current timing uses `time.perf_counter_ns()`. The interval includes fresh initialization, the main loop, output recovery, the batch loop and output append. External comparisons, pre-batch garbage collection, compression and persistence are outside it. GC remains enabled, with thresholds recorded; necessary local checks are not removed. A batch is the statistical unit: dividing its duration by B does not create B independent observations.

Calibration tries $B = 1, 2, 4, 8, 16, 32, 64, 128$ in order, three batches per backend at each attempted value. It chooses the first common B for which both median batch durations reach 10 ms, irrespective of their ratio. All attempts and outputs are retained. At source `c5b5ac3`, the common set selected B=1,2,4,8,16,32 for respectively 24,20,20,28,21,27 inputs; all twelve targeted inputs selected B=8. Independent reconstruction accepted both campaigns, within their separate 600-second budgets. These are granularity decisions, not formal speed comparisons or statistical-precision guarantees.

5.4.1. Scheduling and Noise

Calibration alternates the first backend by case index, candidate index and repeat. The executed formal protocol shuffles cases with seed `20261015`, uses five warm-up calls per backend and twenty paired batches at each frozen B, alternating first position by case index and round. Each backend is first ten times. Failures stop subsequent cases; timeouts preserve committed batches and leave explicit unrun statuses. No outlier removal or speed-based input replacement is allowed.

Historically, each of three algorithms instead received five warm-ups and twenty individually timed calls per input. Case/order seeds were `20261203/20261202`, with cyclic position rotation. The old wrapper disabled GC only around the measured call and restored its prior state; that policy is not inherited by the current comparison. Controlled scheduling reduces fixed-position bias, not all thermal or operating-system noise (Mytkowicz et al. 2009). Repetition alone does not establish adequate statistical power (Kalibera and Jones 2013).

5.4.2. Recorded Environment

Each current archive binds source hashes, Python executable/version/flags, platform, CPU, RAM, power observations, GC thresholds and monotonic-clock metadata. Calibration and formal execution must match source and recorded machine/runtime settings. Tests, qualification and compilation were not run concurrently with calibration or formal sampling. The current formal archives record Apple M4/arm64, 16 GB RAM, ten logical CPUs, macOS 26.6 and CPython 3.12.14; their captured power state is battery at 100%. The historical formal archive records Apple M4/arm64, 16 GB RAM, ten logical CPUs, macOS 26.6 and CPython 3.12.4, with its own preflight; these are not claims about today's runtime.

5.5. Aggregation and Statistical Units

The current formal protocol reports twenty batch-derived per-call durations per input/backend, their median and R7-interpolated quartiles and IQR, and the paired ratio of backend medians. Related constructions and mirrors remain dependent; cross-input summaries are descriptive.

The historical archive contains $60 \times 3 \times 20 = 3600$ measured calls and 180 case–algorithm summaries. Its quartiles use median-of-halves, not R7. The 45 family–size–algorithm groups aggregate case medians; ratios are formed within inputs before equal-case aggregation. Counter associations are exploratory, not causal decompositions of runtime.

5.6. Evidence Binding and Acceptance

The historical formal/pilot sources are `98868b1/01f6480`. The current 140-case and twelve-case qualification producers are `3cafb44` and `668371f`; checker/calibration source is `c5b5ac3`. Formal executions use `4a83fa6`, with identical Python source hashes, frozen B tables and matching required environment fields. Full hashes and immutable archive locations are indexed in section A.2. New checking does not relabel old runs.

Each current record is independently compressed, synchronized and atomically committed. A terminated worker cannot erase earlier completed records. A new archive stores its configuration before execution and binds records, decisions and environment by manifest. Resource wall time includes checking and storage and is separate from a measured call. Old archives are never resealed to imply new execution.

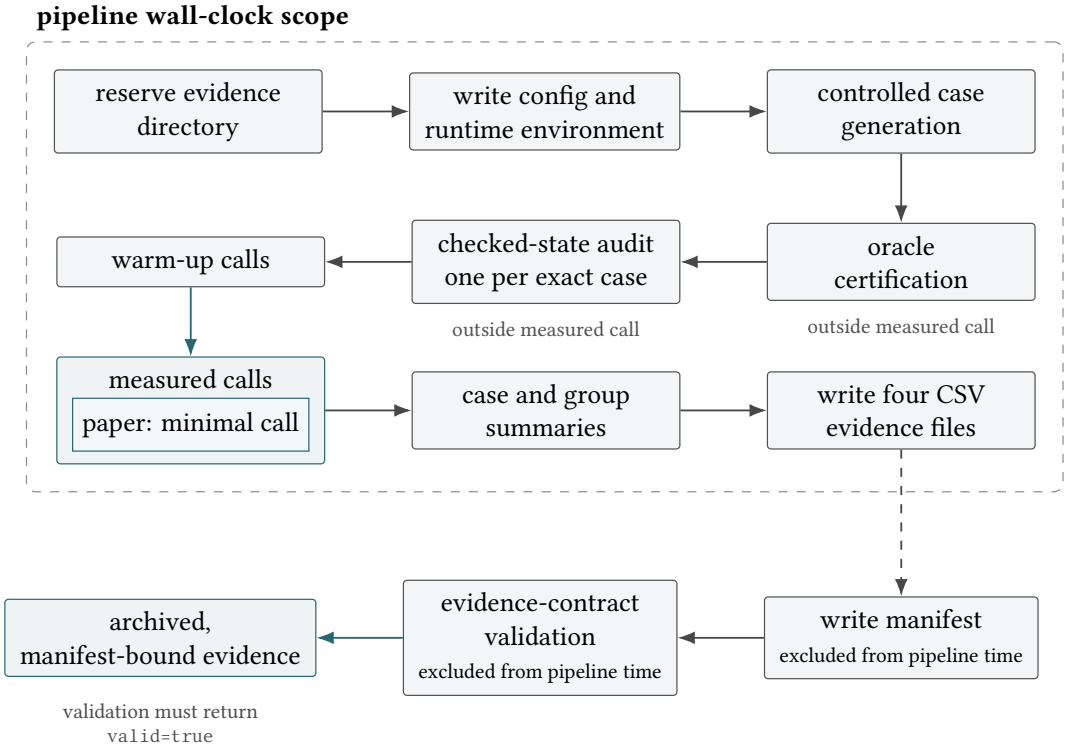


Figure 5.1.: Historical rollback-enabled formal pipeline. Current qualification, batch calibration and formal matched batches have separate identities and schedules.

5.6.1. Fail-Closed Validation

The current archive checker verifies schemas, source blobs, input identities, outputs, event order, sizes, owners, counters and list-ID lifetimes. A live-ID map checks current membership; a separate seen-ID set prevents a singleton or split output from reusing any retired ID. Boundary insertion keeps its live ID. Read-only rechecks accept both existing datasets without rerunning sorting. The measurement validator independently reconstructs candidate order, every attempt, the first qualifying common B and output correctness. For formal archives it reconstructs warm-ups, paired sample order, twenty-batch medians, R7 quartiles and within-input ratios. A process exit code alone is insufficient. Historical validation additionally reran the old checked diagnostics; that behavior is not attributed to the new read-only check.

5.7. Interpretation Boundary

Qualification, calibration, proof review and formal measurement are separate gates. The recorded formal runs followed the source-level manual review and passed their own binding and reconstruction checks. No calibration ratio enters the formal results tables, no historical pilot is renamed as current evidence, and no speed outcome proves the target asymptotic bound.

6. Results

6.1. Current Evidence and Correctness

The current matched experiments completed at source `4a83fa6`, whose Python sources are identical to calibration source `c5b5ac3`. The common and targeted datasets remain separate. Both archives passed independent reconstruction of inputs, configuration, schedule, outputs, frozen batch sizes and statistics. An additional analysis rebuilt the per-input medians and quartiles from every original sample record and matched the validator's values. The manifest-bound records and reproducible analysis are indexed in section A.2.1.

Table 6.1.: Current formal execution. Calls within a batch are not independent samples.

Dataset	Inputs	Sample batches	Sample calls	Warm-up calls
Common	140	5600	62720	1400
Targeted split	12	480	3840	120

Each input/backend contributes twenty sample batches after five warm-up calls. All 66,560 sample calls and 1,520 warm-up calls returned the expected output; no case was erroneous, timed out, skipped or replaced. This output observation is separate from the previously established v2 qualification: formal timing does not repeat checked audits inside the timer. Neither repeated calls nor related constructions create additional independent inputs, and the observed success is finite evidence rather than the general correctness argument.

6.2. Common-Set Runtime and Paired Ratios

The runs use Apple M4/arm64, 16 GB RAM, ten logical CPUs, macOS 26.6 and CPython 3.12.14, with GC enabled at thresholds 700/10/10. The captured power state is battery power at 100%; the records do not establish constant clock frequency or thermal state throughout execution. Both backends use the same public minimal sorting scope, and the tree disables detailed work collection.

Initialization, sorting, output recovery, the batch loop and output append are included; expected-output comparisons and persistence are excluded.

Table 6.2 gives medians across input-specific medians. The last column is instead the median of ratios formed within each input; it is not obtained by dividing the two aggregate time columns. Ratios above one mean a slower tree implementation under this protocol.

Table 6.2.: Common set: descriptive runtime and matched ratios. Times are milliseconds per call; each input first contributes a median of twenty batch-derived observations.

n	Inputs	List (ms)	Tree (ms)	Median ratio
32	14	0.612	1.934	3.153
33	14	0.620	1.957	3.086
64	14	1.291	3.646	3.030
65	14	1.275	4.012	3.089
128	14	2.642	8.082	2.898
129	14	2.772	8.002	2.890
256	14	6.146	13.999	2.625
257	14	6.218	15.427	2.600
512	14	16.390	29.041	2.098
513	14	16.136	33.791	2.123

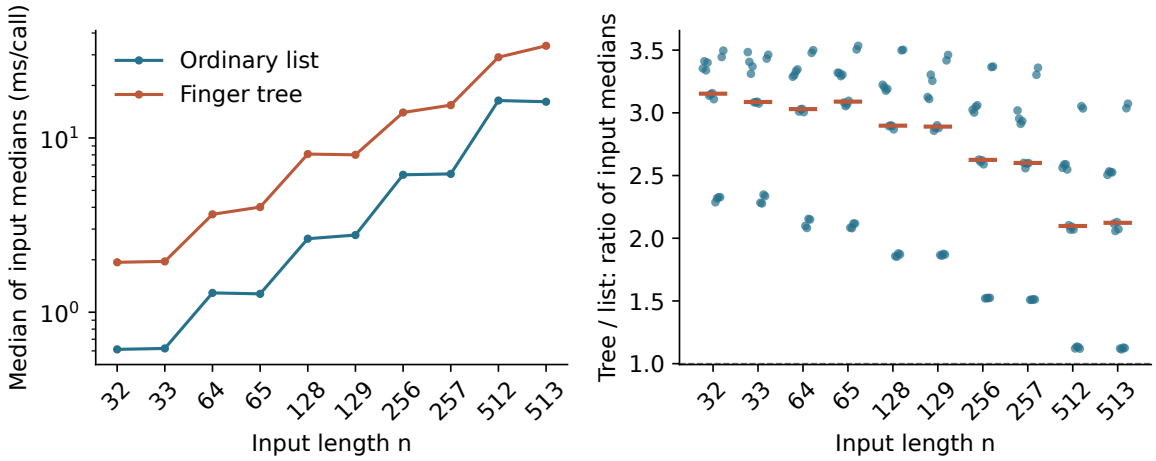


Figure 6.1.: Current common set. Left: medians of input medians, with a logarithmic time axis. Right: all fourteen input ratios per length; horizontal marks are their medians. Horizontal offsets only separate points. The dashed line denotes equal runtime, not a confidence bound.

Across the 140 inputs, tree/list ratios range from 1.114 to 3.535, with median 2.898 and cross-input quartiles 2.117 and 3.210. Every input has a tree median above its matched list median. The size-stratum median decreases from 3.153 at $n = 32$ to 2.123 at $n = 513$, with local nonmonotonic changes. This narrowing gap does not establish a future crossover; no crossover was observed.

6.2.1. Structural Strata

Table 6.3.: Common-set ratios grouped by the frozen nesting classification. These aggregates mix lengths and related constructions.

Stratum	Inputs	Median ratio	Range
Strict-flat	20	3.439	3.035–3.535
Low nesting	40	3.198	2.505–3.484
Medium nesting	40	2.886	2.058–3.157
Heavy nesting	40	1.868	1.114–2.348

The heavy group has the smallest descriptive ratios, but this is not a causal estimate of nesting cost. Constructors change more than one feature and mirror pairs are dependent. In particular, the qualification traces contain 3,788 split calls per backend but no two-sided splits: all take an empty-side outcome. Those executions cover insertion, whole-group movement and ownership updates, not the interior search/dismantle/join path. A nesting label must not be interpreted as a split-work label.

6.3. Independent Targeted Split Set

The twelve targeted inputs at $n = 128, 129$ contain the intended nonempty splits: the smaller side is 3, 7 or 19, each represented by four inputs. Source size is 61 under a finite owner. The traces also cover subsequent retained-side insertion, but not updates to both outputs in every case. Those facts come from qualification traces; formal minimal runs do not collect an intrusive detailed operation ledger.

Table 6.4.: Targeted set, separately analyzed. Each k group has both lengths and both directions.

k	Inputs	List (ms)	Tree (ms)	Median ratio
3	4	2.654	8.432	3.177
7	4	2.673	8.478	3.163
19	4	2.560	8.171	3.207

Ratios range from 3.147 to 3.213; the median is 3.180, with cross-input quartiles 3.165 and 3.189. The tree is slower on all twelve cases. The absence of a monotonic relation with k does not disprove a smaller-side bound: these are complete sorting calls with many other operations, not isolated split latencies. The small targeted set establishes execution of the intended path and a bounded whole-call comparison, not its general performance distribution.

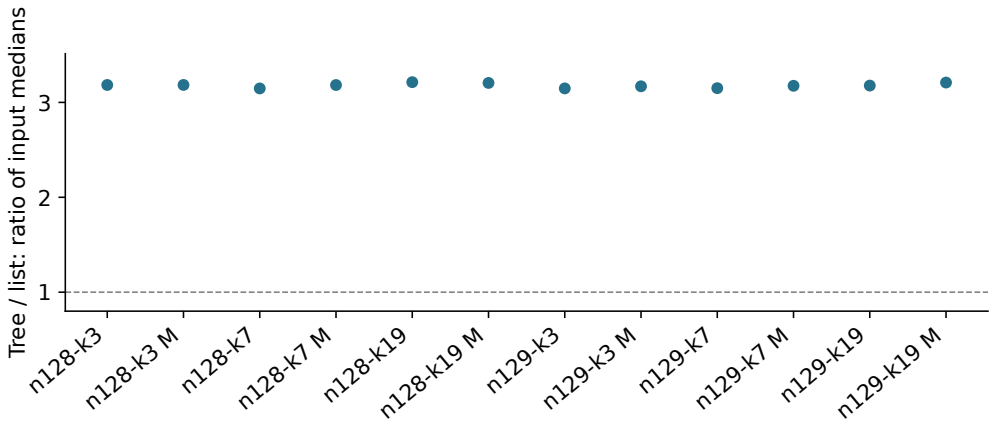


Figure 6.2.: All targeted-set input ratios. M denotes the mirrored construction. Points are ratios of twenty-batch medians, not individual samples or independent random draws.

6.4. Variability and Measurement Accounting

For each input/backend, relative IQR is the R7-interpolated IQR divided by its own twenty-batch median. Across the common set its median is 1.37% for the list and 1.08% for the tree; maxima are 50.26% and 61.81%, respectively. The targeted-set medians are 1.18% and 1.55%, with maxima 9.08% and 7.52%. Thus typical cells have narrow central spreads, but some common-set cells do not. Every batch is retained, including the high-variability cells. No hotspot or thermal experiment was performed, so these variations have no assigned cause.

One common-set sample batch is below 10 ms; none in the targeted set is. The frozen calibration criterion was the median of three calibration batches, not a lower bound on every later batch. B was therefore neither increased nor reselected, and the observation was not excluded. A fixed batch improves clock granularity without guaranteeing stability or statistical precision.

Summed sample intervals are 183.511 seconds for the common set and 21.290 seconds for the targeted set. These are accumulated timed intervals, not campaign wall times: worker startup, GC preparation, validation, compression, synchronization and other orchestration occur outside them. The recorded campaigns remain separate, and their absolute times are not pooled with the older pilot or historical experiments.

6.5. Historical Rollback-Enabled Evidence

The previous study is retained as historical evidence, with its full tables and correlations in the version-bound analysis archive. At source `98868b1`, 60 inputs and three algorithms produced 3,600

measured rows, 180 case summaries and 60 untimed checked-state audits, all successful. Its ordinary-list implementation still performed rollback-related work. Its Python version was 3.12.4; scheduling, GC and quantile conventions differ from the current experiment. No historical observation is included in the tables above.

Table 6.5.: Historical rollback-enabled study only: median within-input paper/reference ratios. The two pipelines have different timed scopes.

n	32	64	128	256	512
Ratio	3.226	2.202	1.351	0.851	0.567

Python sort had the smallest median call time at each historical size. The reference included certification and its complete oracle-backed pipeline, whereas the paper call was pre-certified minimal sorting. The decreasing paper/reference ratio therefore describes those scopes, not an equivalent end-to-end speedup. A separate old pilot reproduced its directional size trend; absolute pilot/formal runtimes were not pooled.

The old exploratory analysis reported positive within-size associations between reference runtime and depth/containment density, with more variable paper-runtime associations. Checked scans, splits and copied/transferred items had positive descriptive associations with minimal runtime. These are neither causal decompositions nor current tree results. The invariant-check counter was constant within size, so its correlation was undefined, not zero. Detailed coefficients remain in the unchanged historical analysis rather than being repeated as evidence about the current implementation.

6.6. Interpretation Boundary

The current comparison supports a clear practical conclusion for the frozen inputs and this environment: the tree implementation is slower despite its reviewed source-level amortized bound. Differing object access, wrappers, local checks, allocation and tree maintenance all contribute to the concrete programs; the experiment does not isolate their individual costs. The data neither refute that structural bound nor demonstrate a practical linear-time speed advantage. Broader scales, other split patterns and another environment remain unmeasured. No causal rollback-removal claim is made.

7. Limitations

7.1. Scope and Version

The current results describe the ordinary-list and finger-tree minimal cores at source `4a83fa6`, with Python sources unchanged from calibration. The earlier rollback-enabled experiment remains separate. The current matched comparison measures those two complete sorting calls; it does not isolate the causal effect of removing rollback. Update failure may leave a partial state, which must be discarded. Successful-state invariants are not an exception-recovery guarantee.

7.2. Mathematical Model and Backend Fidelity

The physical and locality arguments have passed this project's source-level manual review. They have not been machine formalized or independently certified by a separate external reviewer. The account makes endpoint and first-point clarifications explicit, without claiming a unique verbatim interpretation of every omitted paper detail. The finite-parent locality case is a separate project argument; it is not mislabeled as the published latest-child lemma.

The tree's linear total bound uses unit-cost comparisons and machine-word operations with expected dictionary lookup and amortized growth. Arbitrary Python comparison methods, large-number bit costs, interpreter scheduling and wall-clock behavior are outside it. Complete state audits and replay are also outside the minimal complexity claim. A source change can invalidate the review and its binding even if a few tests still pass.

Ordinary lists do not inherit that bound: insertion copying alone is quadratic on increasing inputs, and their membership, ancestry and ownership work differs. The current comparison therefore evaluates whole implementations. It cannot attribute every difference solely to the abstract storage structure, nor use a runtime curve as the proof of either complexity statement.

7.3. Input Coverage and Scale

The common set contains fourteen controlled inputs at each of ten lengths 32/33 through 512/513. Its nesting strata have fixed quotas, and the inputs are constructed rather than sampled from a population of Jordan sequences. Mirror pairs and related constructors are dependent. Consequently, the overall ratio is a descriptive median for this fixed set, not a population performance estimate.

All 3,788 recorded common-set splits per backend have an empty side, so that set does not cover the interior split path. The independent targeted set contains twelve constructions at 128/129, with nonempty smaller sides 3, 7 and 19. Its source size is always 61, its owner is finite, and its later direct updates occur only on the retained side. It does not establish broad coverage of all split ratios, black-height differences or both-output update histories. Earlier operation tests cover other transitions but are not extra performance samples in this study.

No measured input favors the tree. The narrowing common-set gap at larger sizes does not guarantee a crossover beyond 513, or the same behavior for another generator. Theoretical order bounds permit constants that dominate throughout a finite measurement range.

7.4. Timing Scopes and Environment

The current interval includes fresh initialization, the shared algorithm, output recovery and batching overhead. Both backends receive the same scope, but their local checks, field-access wrappers and allocations differ. Their individual contributions were not profiled or experimentally isolated. Expected-output comparison, pre-batch GC collection and evidence persistence are outside timing, while GC remains enabled during sorting.

The formal campaigns record one Apple M4 environment with CPython 3.12.14 and battery power. Warm-ups and alternating first positions reduce selected startup and order effects, but do not establish stable frequency, temperature or absence of all unrelated operating-system activity. The campaigns' separate identities and timestamps are retained. There is no multi-machine replication of the current comparison.

Historical paper/reference ratios have a further scope mismatch: the reference includes certification and its oracle-backed pipeline, while the old paper call is pre-certified minimal sorting. Those ratios are not equivalent end-to-end speedups. Historical CPython 3.12.4 observations, the development pilot and current formal values are not pooled.

7.5. Statistical and Exploratory Reach

Twenty batches characterize repeat variability for each fixed input/backend. Dividing by B gives a per-call duration, not B independent samples. The median and R7 IQR describe those observations; they do not remove persistent bias or provide an independence justification for confidence intervals over related inputs. Cross-input quartiles and ranges are distributions of case summaries, not uncertainty intervals for a universal effect.

Some common-set cells have relative IQR above 50%. They remain included, without a post hoc noise filter, retry or parameter change. One sample batch falls below the calibration target, which applied to three-batch calibration medians rather than every future sample. The exact cause of either observation was not measured. The frozen B table was used throughout.

Old within-size counter correlations are descriptive, based on an unbalanced mix of twelve cases per size. Checked counters and minimal times also come from different observation policies. These correlations cannot causally explain the current tree/list difference. In particular, a constant invariant counter produces an undefined correlation rather than a zero effect.

7.6. Output Provenance and Recognition

The current paper cores traverse their own maintained output list. External certification and expected sorting are used only for qualification and output checking. The distinct reference pipeline deliberately returns oracle-derived output; that provenance must not be attributed to the paper core.

All formal inputs are pre-certified valid. The evidence does not establish recognition accuracy, invalid-input failure reasons or recognition runtime. The sorting proof likewise starts from its validity precondition. Error correction and geometric applications remain outside this evaluation.

7.7. Consequences for the Conclusions

The thesis can report a source-reviewed structural bound in an explicit model, finite qualification and output success, and the measured tree slowdown for two frozen sets. It cannot infer an unconditional Python time bound, a practical speed advantage, independent random sampling, a future crossover, a causal cost decomposition or hardware-independent absolute runtime. These boundaries preserve the distinction between the implemented algorithm, its mathematical account and its measured behavior.

8. Conclusion

8.1. Summary

This thesis turns the 1990 Simplified Jordan Sorting procedure into a shared executable core with two concrete backends. Both maintain their own sorted output and family state; neither receives the oracle's sorted answer. The ordinary-list version makes representation costs visible, including a quadratic insertion-copying lower bound. The real finger-tree version supplies endpoint insertion, path-local split and header ownership transfer while preserving existing pair identities and child groups. Failed updates terminate the call and require the state to be discarded.

The theoretical and empirical contributions have separate support. The source-linked argument reviews reversed-pointer representation, deletion shortness, join ranks, mixed potentials and locality. The matched experiment checks concrete implementations on qualified cases, with frozen batching, recorded environment and independently reconstructed evidence. Neither the proof receipt nor the timing curves substitute for the underlying argument.

8.2. Answers to the Research Questions

8.2.1. RQ1: Executable Reconstruction and Correctness

Both backends execute the same Step 1/2/3 loop and independently recover sorted output. The manual induction explains how neighbor selection, the two odd-stage z_1 corrections, temporary transfer and adoption restore the output and exact family grouping after each completed prefix. The physical argument relates reversed ribs to a logical red–black tree and covers root/nonroot repairs, metadata, endpoints and stable identities.

The two frozen input sets separately passed v2 qualification, which compares independent synchronous checked states, adds a final fresh checked replay and checks a fresh minimal execution. In formal measurement all 66,560 sample calls and 1,520 warm-up calls returned correct outputs, with no failed cases. These finite observations support the implementation; the general argument is manual

and source-specific, not a machine-checked proof or an external proof certification. Valid-input sorting does not establish a recognition algorithm.

8.2.2. RQ2: Observed Runtime Behavior

Under the matched minimal-call scope, the tree is slower on all 140 common inputs and all twelve targeted split inputs. The common-set median of within-input tree/list ratios is 2.898, with range 1.114–3.535; the targeted median is 3.180, with range 3.147–3.213. Size-stratum ratios generally narrow at the larger common lengths, but no measured crossover or guaranteed future crossover follows. Some timing cells have substantial spread and remain in the results without exclusions.

The older rollback-enabled study compared Python sort and two different pipeline scopes. Its numerical results remain historical, as summarized in section 6.5. They are not pooled with this experiment and cannot estimate a causal benefit from removing rollback. The current study answers the backend comparison directly but does not isolate each backend's wrapping, allocation or checking costs.

8.2.3. RQ3: Structure and Implementation Cost

The source analysis supplies amortized constant endpoint insertion and smaller-side logarithmic split using one potential across the mixed sequence. A list-size potential then pays the split terms over the full sorting run. Together with constant stage and ownership access, this gives linear total source-level work under the stated unit-cost and expected/amortized dictionary model. The ordinary-list backend retains its different copying and scan costs.

Measured wall times answer a different question. The common constructions have no two-sided splits; the targeted group supplies that path at smaller sides 3, 7 and 19, but still measures full sorting calls. Neither group identifies an isolated split-cost function. The observed tree slowdown shows that the reviewed operation bound does not ensure a practical advantage at these sizes in this Python implementation. The historical counter correlations remain exploratory associations, not a causal explanation for the current timings.

8.3. Contributions and Remaining Work

The resulting artifact combines an executable reconstruction, a real heterogeneous backend, source-linked mathematical arguments, and a reproducible comparison whose source, inputs, batch choices,

failures and summaries are inspectable. Separating certification, output recovery, physical storage and untimed diagnostics makes both the algorithm and its evidence easier to audit.

Further work should broaden split shapes and lengths under predeclared rules, check another recorded environment, and investigate constant overhead with a separate profiling protocol. Such work should preserve the current negative speed result rather than optimize until a preferred comparison appears. Formalization or independent mathematical review could strengthen the manual argument. Recognition, invalid-input behavior, error correction and polygon clipping require separate specifications and experiments.

8.4. Closing Perspective

The reconstruction now connects the incremental algorithm to the specialized operations it needs, while exposing the limits of a concrete implementation. The structural bound, finite correctness evidence and practical runtime are three different conclusions. Keeping their assumptions and versions explicit allows the thesis to report both the completed backend and its measured slowdown without overstating either result.

A. Claim Traceability

This appendix records how the thesis-facing claim groups connect to the audited claim inventory and archived, manifest-bound experiment evidence. It is a traceability aid rather than an additional source of empirical results. The historical controlled inventory contains 46 claims: nine implementation claims, thirteen method claims, eleven quantitative result claims, four exploratory claims, and nine required limitations. The version-specific amendments below supplement that historical inventory for the revised implementation.

A.1. Claim Groups and Thesis Locations

Table A.1.: Controlled claim groups and their principal locations in the thesis.

Claim IDs	Principal thesis location	Primary authority class
`I-01--I-03`	sections 4.1 and 4.4	Reference and paper-core output provenance in the implementation source and certified-wrapper boundary.
`I-04--I-05`	sections 4.2 and 4.3	Shared Step 1/2/3 runner, partial-order state, and ordinary-list sibling backend.
`I-06--I-08`	sections 4.4 to 4.5	Diagnostics, deterministic replay, fixed execution policies, and pre-call certification.
`I-09`	section 4.6	Dated ordinary-list scope amendment and missing-theoretical-backend boundary.
`M-01--M-08`	sections 5.1, 5.2.1, 5.3, 5.4 and 5.4.1	Pre-specified formal configuration and the reused timing and scheduling control flow.
`M-09--M-13`	sections 5.4.2, 5.5, 5.6 and 5.6.1	Archived manifest-bound summaries, manifest, evidence-contract validator, and recorded execution environment.
`R-01--R-11`	Historical summary in section 6.5; full original tables in the bound analysis archive.	Formal-experiment validated evidence and generated analysis tables; the pilot experiment is used only for the directional replication claim.
`E-01--E-04`	Historical discussion in section 6.5; full coefficient tables retained in the archive.	Generated structure/runtime and checked-counter relationship tables, interpreted descriptively and non-causally.
`L-01--L-09`	chapter 7	Scope amendment, fixed protocol, timing boundaries, sampling design, and the formal analysis limitations.

The detailed wording, evidence locator, and required boundary for every ID are maintained in `docs/thesis/claim_to_evidence_table.md`. The audited copy has the following SHA-256:

```
cc88363a5aa25d55a56b614b02826aa1
862d7c48c8a1a54bd2c66da17715e2ea
```

The chapter-level audit is recorded in `docs/thesis/final_claim_audit.md`; it verifies coverage of all 46 controlled claims and the cross-chapter interpretation boundaries at its historical baseline. It does not certify the later amendments as part of that earlier audit.

A.2. Implementation Revision and Evidence Scope

The subsequent cost correction is recorded separately in `docs/thesis/cost_revision_audit_20261011.md`. It clarifies copying at both insertion boundaries, always-on membership and ancestor scans, and the limited scope of logical split counters. The lower bound in equation (4.1) is derived from the unchanged source, not from the historical timing measurements. The new resource probes are qualification evidence and do not replace any performance results in this thesis.

The earlier revised implementation account is bound to repository commit `6f74683`. The rollback removal was introduced in commit `337903a`; the full identifiers and source-file hashes are recorded in `docs/thesis/revision_claims_20261011.md`. The formal timing archive instead binds source commit `98868b1b705f6d5f22404ee8ad7b88ad7a834f52`. Its results retain their original version and timing scope.

The earlier account of shared-core access interfaces, separate backend-work records and deterministic replay is bound to repository commit `71e3e71`, with evidence in `docs/validation/shared_core_backend_20261012/`. Replay creates a fresh backend of the same type and configuration and compares logical and physical snapshots. This interface revision does not establish completion or performance of the heterogeneous finger-tree backend.

The amendment to historical claim `I-05` replaces the current implementation's rollback contract with error propagation and mandatory discard of failed state. The clarification to `I-07` distinguishes the unchanged execution-policy switches from the changed recovery work. The ordinary-list complexity limitation in `I-09` remains applicable to that backend. The current tree is implemented and qualified. Its representation, mixed-cost and locality arguments have passed the project's source-level manual review under the stated computational model; this is not machine verification or external independent certification.

The current implementation and method amendments bind unchanged production sources at `c5b5ac3`. Tree integration, optional detailed work and v2 validation were introduced in `d6622ce`, `84f7e13`

and `f77df45`. Qualification executions retain producers `3cafb44` (140 common inputs) and `668371f` (twelve targeted split inputs). Read-only ID-lifecycle rechecks and actual calibration use `c5b5ac3`; records, all batch attempts, decisions and independent rebuild reports are in `docs/validation/id_lifecycle_calibration_20261015/`. The later archive commit stores those results without becoming the execution version. The subsequent concentrated review is `docs/development/finger_final_review_20261015.md`; its exact source binding and provenance receipt are in `docs/validation/final_review_formal_20261015/`. The receipt binds the pre-correction review text retained at `ce0148b` and cannot itself prove the mathematics. The October 16 correction makes the red-root recursive join transient explicit and updates its cost-lemma dependencies; its checks are in `docs/validation/join_proof_correction_20261016/`. The old receipt and all measurements remain unchanged. Formal execution at `4a83fa6` followed the original review and passed the separate binding gates; the Python source trees remained identical to `c5b5ac3`.

A.2.1. Current Formal Evidence

The current common and targeted archives, independent validator reports, per-input CSV, full-precision summary and figure-generation program are in `docs/validation/final_review_formal_20261015/`. The raw formal producers are `4a83fa6`; a later commit saving results is not substituted for that identity. The common archive has 5,600 sample batches and the targeted archive 480; both use twenty batches per input/backend and their original calibration B tables. Current numerical claims in the abstract, introduction, results and conclusion derive from those records. Source-level cost claims instead derive from the manual review and its stated model. The directory's dated protocol name is not an execution timestamp: archived UTC timestamps identify the actual campaigns, which span October 15–16 in Europe/Berlin. Old qualification, calibration and historical files are unchanged.

The revision record links these statements to the current split and insertion code, public-call failure tests, and their thesis locations. The historical method, numerical-result, and exploratory claims keep their original evidence bindings. Their counts, tables, figures, and archive hashes are not updated to represent a new experiment. Future implementations and measurements require their own evidence records.

A.3. Archived Evidence Bindings

Each manifest binds six archive files: raw measurements, case summaries, group summaries, case audits, the fixed configuration, and the recorded runtime environment. The separate formal evidence-contract validator regenerates cases and checked diagnostics, recomputes scheduling and summaries,

Table A.2.: Archived run manifests used by the thesis analysis.

Evidence role	Execution identifier	Manifest SHA-256
Pilot experiment	`week11_pilot_v1__run003`	`ef6a9d17df644eab8c3284d04dc356ce4b68b4efdfc0da7eecb5a9c7136c5141`
Formal experiment	`week12_formal_sorting_v1__run001`	`f42ab0b77cb7ea91ed66c66bc6947a81cd8746b5d2b67d526472260d2b47f850`

and verifies these hashes before analysis. The old result figures, retained in the repository, are conversions of historical validated SVG analysis outputs. The five explanatory figures come from committed TikZ sources. Current result plots are generated from the new raw records by the bound analysis program; their sources and generated hashes are indexed in `thesis/figures/README.md` and the current formal archive.

A.4. Interpretation Boundaries

The traceability record preserves the following boundaries across chapters:

- the paper core recovers output from maintained state, while oracle-derived sorted output belongs to the reference pipeline;
- the formal experiment evaluates oracle-certified valid-input sorting, not recognition;
- paper/reference ratios compare different timed pipeline scopes and are not like-for-like end-to-end speedups;
- pilot and formal absolute runtimes are not pooled;
- structure and counter relationships are exploratory and non-causal; and
- the ordinary-list implementation does not inherit the theoretical backend's linear bound; its insertion-copying lower bound is a separate source-derived claim.

Bibliography

Aurenhammer, F. (1987). "Jordan Sorting via Convex Hulls of Certain Non-Simple Polygons". In: *Proceedings of the Third Annual Symposium on Computational Geometry*. SCG '87. New York, NY, USA: Association for Computing Machinery, pp. 21–29. DOI: 10.1145/41958.41961.

Bar-Yehuda, R. and B. Chazelle (1994). "Triangulating Disjoint Jordan Chains". In: *International Journal of Computational Geometry & Applications* 4.4, pp. 475–481. DOI: 10.1142/S0218195994000252.

Berg, M. de, O. Cheong, M. van Kreveld, and M. Overmars (2008). *Computational Geometry: Algorithms and Applications*. 3rd ed. Berlin, Heidelberg: Springer. DOI: 10.1007/978-3-540-77974-2.

Brown, M. R. and R. E. Tarjan (1978). "A Representation for Linear Lists with Movable Fingers". In: *Proceedings of the Tenth Annual ACM Symposium on Theory of Computing*. STOC '78. New York, NY, USA: Association for Computing Machinery, pp. 19–29. DOI: 10.1145/800133.804328.

– (Aug. 1980). "Design and Analysis of a Data Structure for Representing Sorted Lists". In: *SIAM Journal on Computing* 9.3, pp. 594–614. DOI: 10.1137/0209045.

Fung, K. Y., T. M. Nicholl, R. E. Tarjan, and C. J. Van Wyk (June 1990). "Simplified Linear-Time Jordan Sorting and Polygon Clipping". In: *Information Processing Letters* 35.2, pp. 85–92. DOI: 10.1016/0020-0190(90)90111-A.

Hoffmann, K., K. Mehlhorn, P. Rosenstiehl, and R. E. Tarjan (1985). "Sorting Jordan Sequences in Linear Time". In: *Proceedings of the First Annual Symposium on Computational Geometry*. SCG '85. New York, NY, USA: Association for Computing Machinery, pp. 196–203. DOI: 10.1145/323233.323259.

– (1986). "Sorting Jordan Sequences in Linear Time Using Level-Linked Search Trees". In: *Information and Control* 68.1–3, pp. 170–184. DOI: 10.1016/S0019-9958(86)80033-X.

Huddleston, S. and K. Mehlhorn (1982). "A New Data Structure for Representing Sorted Lists". In: *Acta Informatica* 17.2, pp. 157–184. DOI: 10.1007/BF00288968.

Kalibera, T. and R. E. Jones (2013). "Rigorous Benchmarking in Reasonable Time". In: *Proceedings of the 2013 International Symposium on Memory Management*. ISMM '13. New York, NY, USA: Association for Computing Machinery, pp. 63–74. DOI: 10.1145/2464157.2464160.

McGeoch, C. C. (2012). *A Guide to Experimental Algorithmics*. Cambridge: Cambridge University Press. DOI: 10.1017/CBO9780511843747.

Mytkowicz, T., A. Diwan, M. Hauswirth, and P. F. Sweeney (2009). "Producing Wrong Data Without Doing Anything Obviously Wrong!" In: *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS XIV. New York, NY, USA: Association for Computing Machinery, pp. 265–276. DOI: 10.1145/1508244.1508275.

Sanders, P. (2009). "Algorithm Engineering: An Attempt at a Definition". In: *Efficient Algorithms: Essays Dedicated to Kurt Mehlhorn on the Occasion of His 60th Birthday*. Ed. by S. Albers, H. Alt, and S. Näher. Vol. 5760. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, pp. 321–340. DOI: 10.1007/978-3-642-03456-5_22.

Sojka, E. (Oct. 1997). "Two Simple and Efficient Algorithms for Jordan Sorting and Polygon Cutting and Clipping". In: *Computer Networks and ISDN Systems* 29.14, pp. 1661–1673. DOI: 10.1016/S0169-7552(97)00081-0.

– (Apr. 1999). "A New Algorithm for Jordan Sorting: Its Average-Case Analysis". In: *Discrete Applied Mathematics* 93.1, pp. 127–140. DOI: 10.1016/S0166-218X(99)00011-6.

Tarjan, R. E. and C. J. Van Wyk (Feb. 1988). "An $O(n \log \log n)$-Time Algorithm for Triangulating a Simple Polygon". In: *SIAM Journal on Computing* 17.1, pp. 143–178. DOI: 10.1137/0217010.